As we mentioned at the beginning of this section, we are using MLflow's Evaluate capability to facilitate our model evaluation. Before we evaluate our RAG chatbot, let's load the methods and verify how MLflow Evaluate works by default. We first load the "answer correctness" metric, which we will use as-is.

```python
from mlflow.metrics.genai.metric_definitions import answer_correctness
from mlflow.metrics.genai import make_genai_metric, EvaluationExample

# Because we have our labels (answers) within the evaluation dataset, we can evaluate the answer correctness as part
of our metric. Again, this is optional.
answer_correctness_metrics = answer_correctness(model=f"endpoints:/{endpoint_name}")
print(answer_correctness_metrics)
```

```
Example Input:
How is MLflow related to Databricks?

Example Output:
MLflow is a product created by Databricks to enhance the efficiency of machine learning processes.

Additional information used by the model:
key: targets
value:
MLflow is an open-source platform for managing the end-to-end machine learning (ML) lifecycle. It was developed by Databricks, a
company that specializes in big data and machine learning solutions. MLflow is designed to address the challenges that data scie
ntists and machine learning engineers face when developing, training, and deploying machine learning models.

Example score: 4
Example justification: The output provided by the model is mostly correct. It correctly identifies that MLflow is a product crea
ted by Databricks. However, it does not mention that MLflow is an open-source platform for managing the end-to-end machine learn
ing lifecycle, which is a significant part of its function. Therefore, while the output is mostly accurate, it has a minor omiss
ion, which is why it gets a score of 4 according to the grading rubric.
```

Figure 7.52 – Viewing the answer correctness metrics

The out-of-the-box metrics are good, but professionalism is also an important criterion for our use case, so we create a custom professionalism metric. *Figure 7.53* shows how you can use the `make_genai_metric()` function to build out our professionalism evaluation metric.

```python
professionalism = make_genai_metric(
    name="professionalism",
    definition=(
        "Professionalism refers to the use of a formal, respectful, and appropriate style of communication that is "
        "tailored to the context and audience. It often involves avoiding overly casual language, slang, or "
        "colloquialisms, and instead using clear, concise, and respectful language."
    ),
    grading_prompt=(
        "Professionalism: If the answer is written using a professional tone, below are the details for different scores: "
        "- Score 1: Language is extremely casual, informal, and may include slang or colloquialisms. Not suitable for "
        "professional contexts."
        "- Score 2: Language is casual but generally respectful and avoids strong informality or slang. Acceptable in "
        "some informal professional settings."
        "- Score 3: Language is overall formal but still have casual words/phrases. Borderline for professional contexts."
        "- Score 4: Language is balanced and avoids extreme informality or formality. Suitable for most professional contexts. "
        "- Score 5: Language is noticeably formal, respectful, and avoids casual elements. Appropriate for formal "
        "business or academic settings. "
    ),
    model=f"endpoints:/{endpoint_name}",
    parameters={"temperature": 0.0},
    aggregations=["mean", "variance"],
    examples=[professionalism_example],
    greater_is_better=True
)
```

Figure 7.53 – Adding a custom professionalism metric